hashlock.

2025

Penetration Testing

DEDCAT (DedcatBounce)

November 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The DEDCAT team partnered with Hashlock to conduct a security Penetration Test of their DedcatBounce Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

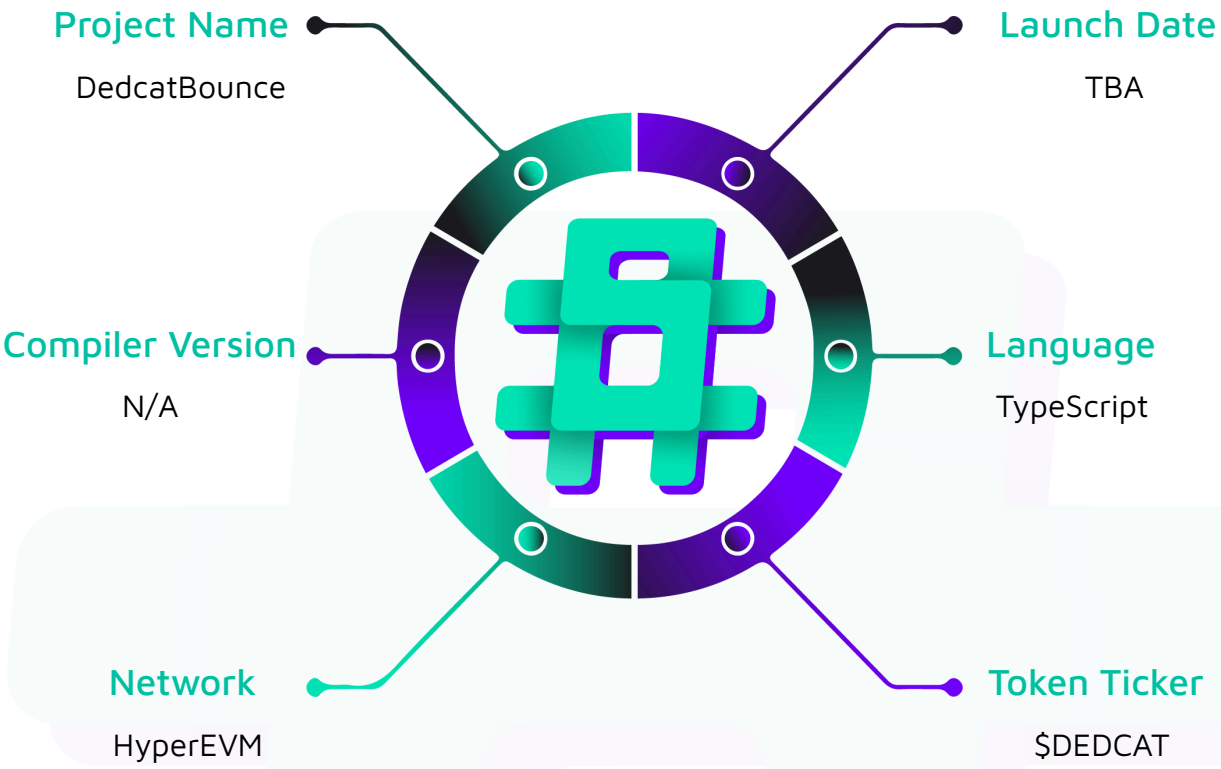
DedcatBounce is a last-player-standing game on HyperEVM. Players spend $HYPE to Bounce the cat and reset a live countdown timer. When the timer inevitably hits zero, the last player to Bounce wins the prize pool.

Each Bounce also fuels a referral system, where active players earn rewards from the Bounces made by others they referred into the game. This creates a network of competitive, self-propelling communities known as Bounce Cabals, driving both engagement and growth.

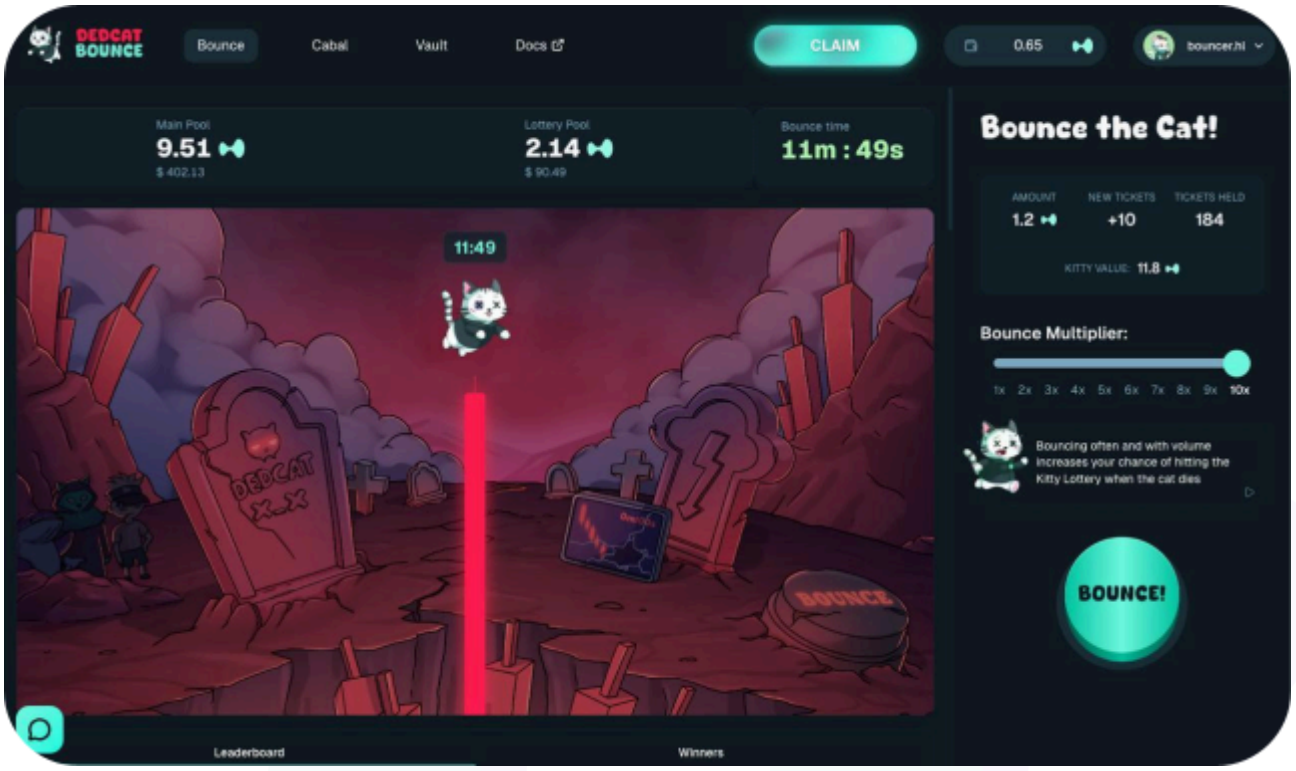
DedcatBounce is the first dApp in the Dedcat ecosystem — a composable treasury of on-chain games and DeFi primitives, all routing revenue back to $DEDCAT stakers. It serves as the ignition point for Dedcat's flywheel, proving how social gameplay and real yield can power a self-sustaining community owned on-chain economy.

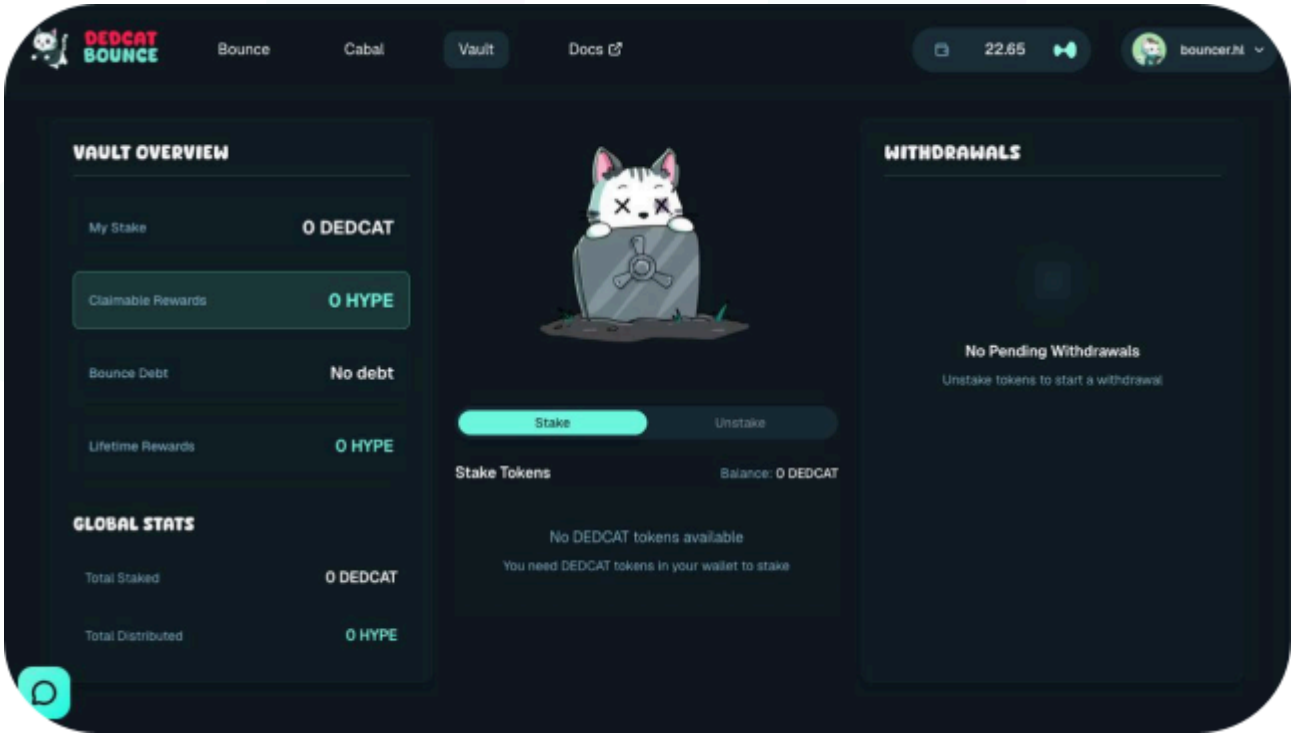
Project Name: DedcatBounce
Project Type: dApp
Website: ded.cat
Logo:

Visualised Context:



Project Visuals:





Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the DedcatBounce project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	**DedcatBounce Project code base**
Platform	**Web apps & APIs**
Penetration Test Date	**November, 2025**
Repository/Website URL/IP Tested	https://github.com/DedcatBounce-inc/DedcatBounce
Component	Privy integration - authentication and authorization, best practises

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Penetration Test Findings section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

3 Medium-severity vulnerabilities

4 Low-severity vulnerabilities

3 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system’s performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

Medium

[M-01] Badges#submitTwitterPost - Weak URL validation allows bypass

Description

The Twitter `URL` validation in `submitTwitterPost` uses string inclusion checking which can be bypassed by including the domain anywhere in the `URL` string.

Vulnerability Details

The validation logic only checks if the `URL` contains `"x.com"` or `"twitter.com"` as substrings, allowing malicious URLs like `https://evil.com?dummy=x.com&status=` to pass validation.

```
if (
  !args.twitterUrl.includes("x.com") &&
  !args.twitterUrl.includes("twitter.com")) {
  throw new ConvexError({ type: ErrorType.INVALID_INPUT })}
```

Impact

Users can submit `non-Twitter` `URLs` for badge verification, potentially gaming the badge system or causing unexpected behavior in processing.

Recommendation

Implement proper `URL` parsing and validate the hostname exactly matches `"x.com"` or `"twitter.com"` using `URL` constructor and hostname property.

Status

Resolved

[M-02] Init#initializeBadgesManual - Unauthenticated badge initialization causes data integrity issues

Description

The `initializeBadgesManual` mutation lacks authentication checks and can be called repeatedly, potentially corrupting badge definitions or causing duplicate entries.

Vulnerability Details

The function is exposed as a public mutation without authentication or idempotency checks. Multiple calls could create duplicate badge entries or interfere with legitimate badge initialization.

```
export const initializeBadgesManual = mutation({
  args: {},
  handler: async ctx => {
    await ctx.runMutation(internal.badges.initializeBadges)
    return "Badges initialized successfully"
  },
})
```

Impact

Unauthenticated users can spam badge initialization causing database bloat or corrupted badge definitions.

Recommendation

Add authentication checks and prevent duplicate initialization, or remove the function entirely if initialization is handled elsewhere.

Status

Resolved

[M-03] MessageReactions#toggleReaction - Emoji validation bypass allows invalid reactions

Description

The emoji validation uses `includes()` which allows strings containing valid emojis plus additional characters to pass validation checks.

Vulnerability Details

The validation check `ALLOWED_EMOJIS.includes(args.emoji)` will incorrectly validate multi-character strings if they contain an allowed emoji as a substring, permitting invalid reactions like "👍malicious" when only "👍" should be allowed.

```
if (!ALLOWED_EMOJIS.includes(args.emoji)) {

  throw new Error("Invalid emoji")

}
```

Impact

Users can create reactions with arbitrary text appended to valid emojis, potentially causing display issues or unexpected behavior in the UI.

Recommendation

Replace `includes()` check with exact string matching using `ALLOWED_EMOJIS.indexOf(args.emoji) !== -1 or ALLOWED_EMOJIS.some(e => e === args.emoji)`.

Status

Resolved

Low

[L-01] Cabal#validateReferralCode - Referral code can be bruteforced

Description

The `validateReferralCode` query lacks rate limiting protection, allowing attackers to brute force valid referral codes without restriction. The function explicitly notes that rate limiting cannot be applied to query functions. The comment acknowledges this limitation but accepts the risk without implementing alternative protections.

Recommendation

If rate limiting is not possible, enforce referral code complexity.

Status

Acknowledged

[L-02] Admin#clearAllTables - Unused database clearing function

Description

The `clearAllTables` function in `admin/maintenance.ts` iterates through all application tables and deletes every document. While marked as `internalMutation`, keeping unused destructive operations in production code violates the principle of least privilege, creating unnecessary risk if accidentally exposed or called.

Recommendation

Remove the `clearAllTables` function entirely from the codebase if not required for legitimate maintenance operations.

Status

Resolved

[L-03] Error#handleConvexError - Verbose errors may expose sensitive information

Description

The `handleConvexError` function in `app/convex/lib/errorHandler.ts` uses `console.error` to log full error details including stack traces. It is used to handle the application errors. In browser environments, these logs could expose internal implementation details, file paths, and system architecture information. Additionally, all the errors are rethrown by the handler.

Recommendation

Display only generic error messages to application users without any technical details inside.

Status

Resolved

[L-04] Users#ensureUser - Public mutation enables spam attacks

Description

The `ensureUser` mutation is exposed as a public endpoint when it should be internal, allowing unauthenticated repeated calls that consume rate limits and database resources.

The function is defined as a public `mutation` rather than `internalMutation`, exposing it to external calls. While rate limiting exists, unnecessary public exposure increases attack surface for resource exhaustion.

Recommendation

Change `ensureUser` from `mutation` to `internalMutation` to restrict access to internal callers only.

Status

Acknowledged- This is one of the recommended ways of storing user information by Convex https://docs.convex.dev/auth/database-auth#call-a-mutation-from-the-client

it is minimized by being an authedMutation and it is rate limited

QA

[Q-01] HTTP#test - Leftover test endpoint in production code

Description

The `/test` endpoint in `http.ts` appears to be debugging code that returns a simple status message.

Recommendation

Remove the test endpoint from production builds.

Status

Resolved

[Q-02] Messages#listMessages - Anyone can read all messages

Description

While reading all messages appears to be a public chat feature, they could be secured by enforcing authentication.

Recommendation

Informational only, it may be a design decision.

Status

Acknowledged - intended behaviour chat is public, writing messages however is not.

[Q-03] API - Missing HTTP headers

Description

The application does not implement `HTTP` headers to increase security - `X-Content-Sniffing`, `HSTS` and Content Security Policy. While attacks are mitigated to some point by native React `HTML` sanitization, it is still recommended to add additional headers to improve security.

Recommendation

Implement security headers to the application. Pay attention to integrating it properly with privy or other external integrations.

Status

Acknowledged

Conclusion

After Hashlock's analysis, the DedcatBounce project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved or acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au